

Typer, operatorer och uttryck

Variabelnamn

Variabelnamn kan inte se ut riktigt hur som helst, det finns vissa restriktioner för dem. De måste bestå av bokstäver och siffror, och första tecknet måste vara en bokstav. (Även om ”_” inte är en bokstav, räknas den i detta sammanhang som en sådan.) Versaler och gemener skiljs åt, vilket innebär att N och n är två olika namn på variabler. Som i de flesta språk är nyckelord såsom if och int reserverade: de kan inte användas som variabelnamn.

Datatyper och storlekar

De grundläggande datatyperna i C är:

char	character, tecken
int	integer, heltal
float	flyttal, enkel precision
double	flyttal, dubbel precision

Dessa kan utökas på exempelvis följande sätt:

```
short int
long int
signed int
unsigned int

signed char
unsigned char
```

Ett tecken (char) skrivs som ett tecken inom ’ ’. En sträng är en sekvens av tecken (char) inom ” ”. Tomma strängen skrivs ””. Rent tekniskt set är en sträng ett fält av char. Den interna representationen av en sträng är nollterminerad ’\0’. Detta innebär att ’a’ inte är det samma som ”a”.

Deklarationer

Alla variabler måste deklarerars före de används. En deklareringspecificerar typ, och innehåller en lista av en eller flera variabler av samma typ, dvs.

```
typ var1, var2, ... , varN;
```

Exempel

```
int i, j max, min;
float ang;
```

En variabler kan också initieras när den deklarerars. Om namnet följs av ett likhetstecken och ett uttryck, initieras variabeln till detta uttryck.

```
typ var = uttryck;
```

Exempel

```
int min = 0;
```

const kan läggas till deklareringsen för att specificera att variabelns värde inte kommer ändras

```
const typ var = uttryck;
```

Exempel

```
const int fieldlength = 10;
```

Om en konstant finns på många ställen i ett program och denna konstant ofta behöver ändras är det praktiskt att definiera den i början av programmet på följande sätt

```
#define uttryck1 uttryck2
```

Exempel

```
#define fieldlength 10
```

Operatorer

Aritmetiska operatorer

+	Addition
-	Subtraktion
*	Multiplikation
/	Division
%	Modulo

Relations operatorer

>	Större än
>=	Större än eller lika
<	Mindre än
<=	Mindre än eller lika

Likhets operatorer

==	Lika
!=	Inte lika

Logiska operatorer

&&	Och
	Eller
!	Invertera

Bit operatorer

&	Och
	Eller
^	XOR
<<	Vänsterskift
>>	Högerskift
~	Komplement

Övriga operatorer

++	Öka operator, adderar 1 till operanden
--	Minska operator, subtraherar 1 från operanden

Om operatören står före en variabel, ökas respektive minskas dess värde före det används. Om operatören istället står efter variabeln ökas/minskas den efter dess värde använts.

Tilldelningsoperatorer och uttryck

Uttryck såsom

```
i = i + 2;
```

där variabeln i vänstra ledet upprepas direkt i det högra kan skrivas

```
i += 2;
```

Operatorn += kallas tilldelningsoperator. De flesta binära operatorer har en motsvarande operator op=, där op är någon av +, -, *, /, %, <<, >>, &, ^, |. Det vill säga

```
uttryck1 = uttryck1 op uttryck2;
```

är ekvivalent med

```
uttryck1 op= uttryck2;
```

Den treställiga operatorn uttryck1 ? uttryck2 : uttryck3, gör att uttryck såsom

```
if ( a > b)
  z = a;
else
  z = b;
```

kan skrivas

```
z = (a > b) ? a : b;
```

uttryck1 evalueras först, om det är skilt från noll evalueras uttryck2. Om inte, evalueras uttryck3.

Flödeskontroll

Med hjälp av olika uttryck och tester kan man påverka vilka satser som skall utföras, och i vilken ordning de skall tas. Vi har redan tittat på de flesta (if, for, while mm), och här kommer en liten fördjupning.

if – villkorsstyrda beslut

I if-satsen har vi möjligheten att påverka vilka instruktioner datorn skall utföra. Satsen fungerar precis som svenskan (och engelskan), dvs:

Om det är så här, gör detta. Annars gör det här.

I C skrivs en if-sats som följer:

```
if(villkor1)
    sats1/block1
else if(villkor2)
    sats2/block2
else
    sats3/block3
```

Om *villkor1* är uppfyllt kommer *sats1/block1* att köras. *else if(villkor2)* kommer ej att testas, och varken *sats2/block2* eller *sats3/block3* kommer att utföras.

Är däremot inte *villkor1* uppfyllt, testas *else if* – satserna, ända tills ett villkor är uppfyllt. *else*-satsen utförs de gånger som varken *if*- eller *else if*- satserna utförts.

Man kan ha flera *else if*-satser om man vill. Man behöver ej heller ha någon. *else*-satsen kan också utelämnas.

Ex:

```
if (betyg == 5)
    printf("Mycket bra");
else if(betyg == 4)
    printf("Ah, fint");
else if(betyg == 3)
    printf("Ok");
else
    printf("Illa");
```

switch

Istället för att, som i exemplet ovan, använda en rad *else if*-satser kan man ibland använda sig av en *switch* istället.

```
switch(uttryck){
    case konstant1: sats1/block1
    case konstant2: sats2/block2
    default: sats3;
}
```

Viktigt att tänka på när man använder en *switch* är att varje *sats/block* skall avslutas med en ”*break*”, annars kommer flera ”*case*” att utföras (alla de som ligger efter det ”rätta”).

Om vi tittar på exemplet med *if – else if – else* ovan, kan vi skriva detta som en *switch*-sats på följande vis:

```
switch(betyg){
    case 5:
        printf("Mycket bra");
        break;
    case 4:
        printf("Ah, fint");
        break;
    case 3:
        printf("Ok");
        break;
    default:
        printf("Illa");
        break;
}
```

Loopar – while och do while

Ofta vill man att ett antal satser skall upprepas och utföras flera gånger. För att på ett enkelt sätt göra detta i C kan man använda sig av loopar. *while*-loopen utför en sats eller ett block så länge ett villkor är uppfyllt. Även här kan översättning till svenska beskriva precis vad det är som händer: Så länge detta gäller, gör så här.

För att använda en *while*-sats, skriv:

```
while(villkor){
    sats/block
}
```

sats/block kommer nu att utföras så länge villkoret är uppfyllt.

Ex:

```
unsigned int i = rand();
while( i == 12 || i == 27 || (i > 765 && i < 1000))
    i = rand();
```

I programsekvensen ovan kommer *i* att få ett slumpvalt värde. Sedan vill vi ej att *i* skall ha värdet 12, 27 eller något tal mellan 765 och 1000, och vi slumpar därför fram ett nytt *i* så länge *i* är detta.

Man kan också ”vända på” *while*-satsen, och skriva villkoret sist:

```
do{
    sats/block
}while(villkor)
```

Den stora skillnaden är att sats/block kommer att utföras en gång, oavsett om villkoret är uppfyllt. I exemplet ovan skulle det till exempel vara användbart, då vi ju ville utföra raden

```
i = rand();
```

minst en gång.

Loopar – for

Det är mycket vanligt att man vill upprepa en sats eller ett block ett visst antal gånger. Det kan till exempel vara de gånger som man vill gå igenom alla element i ett fält. Detta kan givetvis göras med en *while*-slinga, men då det är en så vanlig procedur finns det ett bättre sätt:

```
for(uttryck1; villkor; uttryck2){  
    sats/block  
}
```

där *uttryck1* är en sats som kommer att utföras första gången, innan loopen startar. Villkoret testas därefter, för att se om loopen skall tas. Om loopen tas kommer *uttryck2* att utföras nästa varv, och så fortsätter det tills villkoret inte längre är uppfyllt.

Ex.

```
char[][] monader = {"Jan", "Feb", "Mar", "Jun", "Jul", "Aug", "Sep", "Okt", "Nov",  
"Dec"};
```

```
int i;  
for(i = 0; i < 12; i++){  
    printf("%s\n",monader[i]);  
}
```

I loopen ovan kommer *i* först att sättas till 0. Sedan testas villkoret: *i* är mindre än 12, så raden *printf(...)* utförs. Därefter ökas *i* med ett (*i* = 1), och villkoret testas igen. *i* är fortfarande mindre än 12, och *printf(...)* utförs igen. Så fortsätter det tills *i* är 12, då villkoret inte längre är uppfyllt, och loopen slutar.

Mer om loopar - break och continue

Om man vill avbryta en pågående loop kan man använda *break*. *break* gör att exekveringen av programmet fortsätter efter loopen.

continue kan användas för att hoppa över resten av satserna i loopen (som om de inte fanns) och återgå till loopens början.

Ex:

```
int[] lista = {1,2,3,-4,5,0,6};  
int i;  
for(i = 0; i < 7; i++){  
    if(lista[i] < 0)  
        continue;
```

```

        else if(lista[i] == 0)
            break;
        printf("%d ", lista[i]);
    }

```

Programmet kommer att skriva ut de tal som står i fältet lista, men hoppa över de som är negativa, och avbryta när en nolla dyker upp. Utskrift: 1 2 3 5

Goto och Labels

I C finns också möjligheten att använda *goto* och *labels*. Med en *label* (etikett) kan man "namnge" en plats i programmet, och med *goto* kan man sedan hoppa hit.

Ex:

```

start:
    printf("HEJ\n");
    goto start;

```

Programmet kommer att skriva ut en massa HEJ på skärmen, tills användaren tröttnar och dödar processen.

Generellt kan man säga att *goto* och *labels* bör undvikas. Programmet blir mycket mer svårläst, och det är lätt att göra fel. Det går lika bra att klara sig utan *labels* som med, men C ska ju vara ett kraftfullt språk, och det är upp till användaren att bestämma hur han/hon vill ha det.

Funktioner och programstruktur

Funktioner delar in program i mindre delar, och låter programmerare ta del av vad andra redan gjort genom bibliotek som innehåller funktioner. Bra funktioner gömmer implementationsdetaljer för andra delar av programmet som inte behöver veta om dessa. Detta gör det lättare att se helheten samt lättare att ändra och underhålla koden.

Syntaxen för funktioner är som följande:

```
return_type function_name ( argument declarations )
{
    declarations and statements;

    return expression;
}
```

t.ex. :

```
void error(char *msg)
{
    printf("Error: %s\n", msg);
}

int shift_left(int num, int sh)
{
    int ret = num << sh;

    return ret;
}
```

Variabler – static, extern, const & register

Variabler deklareras som sagts innan i början av block och deras räckvidd är inom det blocket (t.ex. variablerna deklarerade i början av en funktion) de är deklarerade.

```
{
    deklaration av variabel
    satser
} // slut på variabelns räckvidd
```

Om ett variabelnamn används flera gånger gäller den “innersta” variabeln, t.ex:

```
void do_nothing(int num)
{
    int x = 50, i;

    if(num == 0){
        int x = 6;
        printf("%d\n", x);
    }
}
```

Detta kommer leda till en utskrift av “6” om *do_nothing* anropas med '0'. Externa variabler, är variabler som är deklarerade globalt. Dessa variabler deklareras i samma block som funktioner. Deras räckvidd gäller nedanför den plats de deklareras, för åtkomst av globala variabler i andra filer använder man nyckelordet *extern*.


```
fil1.c
int chk = 5;
void func();
```

```
fil2.c
extern int chk; // nedan kan funktioner använda chk
```

Om man vill ha en global variabel oåtkommlig för andra filer använder man *static*:

```
static typ namn;
```

Detta får till följd att endast källfilen *namn* är deklarerad i har åtkomst till den. Ett sätt att minimera kontaktytor mellan olika funktioner, om man ändå måste använda globala variabler.

Nyckelordet *static* har en helt annan effekt om det används på variabler som inte är globala. Dessa blir då “statiska” i den meningen att variabeln inte förstörs efter funktionen körts, t.ex. :

```
void counter()
{
    static int count = 0;
    printf("Den här funktionen har anropats %d ggr.\n", ++count);
}
```

Vidare finns *const*, *register* & *volatile*, som appliceras på samma sätt som *static*:

```
modifierare typ namn;
```

En variabel deklarerad som *const* kan inte ändras. Det kan även användas vid funtionsargument och ger då extra information om funktionen inte har för avsikt att ändra indata. Pekare kan även deklarerars *const*:

```
int const *a;
int *const b;
int *const *c;
```

Här är “a” deklarerad att vara en “pekare till en *const* int”, “b” är “*const* pekare till en int”, det beror alltså var man sätter *const*. Det förbryllande “c” är en “pekare till en *const* pekare till en int”.

Genom att deklarera en variabel *register* ger man kompilatorn ett tips att denna variabel kan vara bra att ha i ett cpu register. Detta är dock bara ett tips eftersom kompilatorn kan strunta i det helt.

Volatile betyder att variabeln ifråga kan ändras av källor utom programmets kontroll, som t.ex. minnesmappad io. Detta gör att kompilatorn aldrig antar något om variabelns värde och optimerar så gamla värden används (genom att lägga i register t.ex.).

Header filer och uppdelning i olika filer

Man kan dela upp program i flera filer och kan på så sätt få mer lätthanterlig källkod och samtidigt slippa kompilera om onödig kod som inte ändrats. Filer med deklARATIONER kallas *headerfiler* och har suffixet *.h*, dessa inkluderas i källkod genom *#include*. Vanlig c källkod brukar ha suffixet *.c*. Genom uppdelning kan filer kompileras separat och sedan länkas ihop till ett program.

```
[uffe@waagh project]$ gcc -c main.c
[uffe@waagh project]$ gcc -c sort.c
[uffe@waagh project]$ gcc main.o sort.o -o sort
```

Ovanstående kompilerar *main.c* och *sort.c* separat och länkar sedan dem. Headerfilerna används för att visa vad för funktioner man ska använda i *.c* filen.

Rekursion

I C kan man anropa en funktion inom sig själv.

```
int stoopid_sum(unsigned int n)
{
    if (n = 1)
        return 1;

    return n + stoopid_sum(n-1);
}
```